

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	D.Raghava
Name	192525304

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I D.Raghava(192525304) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.Raghava

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and	Most constraints identified	Some constraints identified	Constraints poorly understood

		prioritized accurately			
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Q1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

Answer

A media platform stores large amounts of videos, images, audio files, metadata, and user-generated content. HDFS is suitable because it provides distributed storage and fault tolerance across multiple DataNodes.

1. Constraints

The major constraints are:

- **Data availability > 99.9%**
- **Replication factor ≤ 3**
- **Cost-efficient storage growth**
- Ability to handle continuously increasing media data
- Protection against DataNode failures

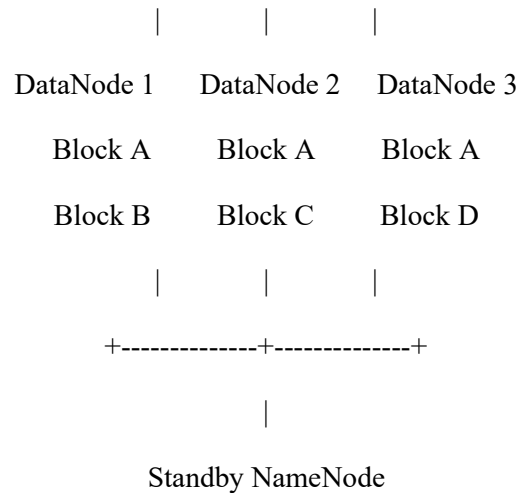
2. Proposed HDFS Architecture

The platform can use:

- **1 Active NameNode**
- **1 Standby NameNode**
- Multiple **DataNodes**
- HDFS replication factor of **3** for critical data
- Replication factor of **2** for less-critical or easily reproducible data where appropriate
- Rack-aware block placement
- Automatic rebalancing and capacity monitoring

Basic architecture:





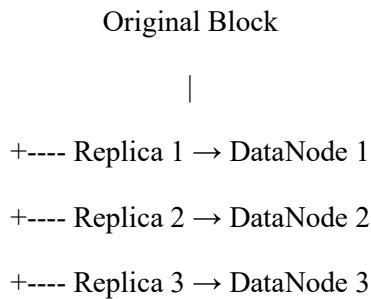
3. Replication Strategy

HDFS divides large files into blocks and stores multiple copies of each block.

For important media data, use:

Replication Factor = 3

For example:

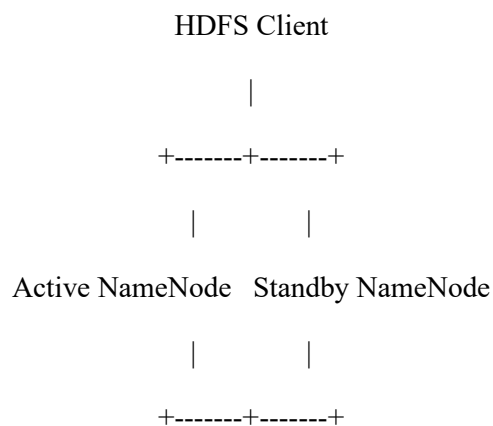


If one DataNode fails, another replica can serve the data.

The replication factor is kept at **3 or below**, satisfying the constraint.

4. High Availability

To maintain availability above 99.9%, use an **Active–Standby NameNode architecture**.



|

DataNodes

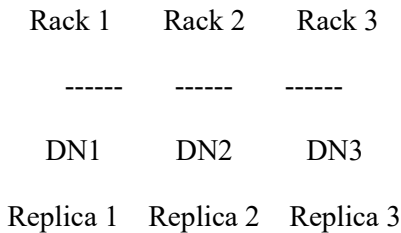
If the Active NameNode fails, the Standby NameNode can take over.

DataNodes also maintain multiple copies of blocks, so failure of individual storage nodes does not necessarily make data unavailable.

5. Rack Awareness

Data replicas should not all be stored on the same physical rack.

For example:

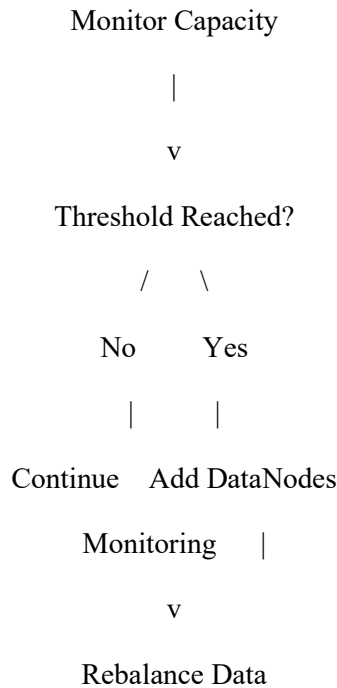


This protects against rack-level failures.

6. Cost-Efficient Growth

Storage should be expanded incrementally instead of purchasing excessive hardware in advance.

When storage utilization approaches a predefined threshold:



Older and less frequently accessed media can also be moved to lower-cost storage tiers where the overall architecture permits it.

7. Data Compression

Compression can reduce storage requirements.

Examples include:

- Video compression
- Gzip for suitable text/log data
- Columnar formats such as Parquet for analytical metadata

Compression reduces the amount of physical storage required and therefore lowers infrastructure cost.

8. Monitoring

The platform should continuously monitor:

- DataNode health
- Disk utilization
- Missing or under-replicated blocks
- Network usage
- NameNode health
- Storage growth

Automatic re-replication can restore the required number of replicas after a failure.

Conclusion

The proposed HDFS architecture uses **Active–Standby NameNodes, multiple DataNodes, rack-aware replication, controlled replication factors, monitoring, compression, and incremental capacity expansion**. A maximum replication factor of 3 provides fault tolerance without excessive storage overhead. Therefore, the design can achieve the required **>99.9% availability while maintaining cost-efficient storage growth**.

Q2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

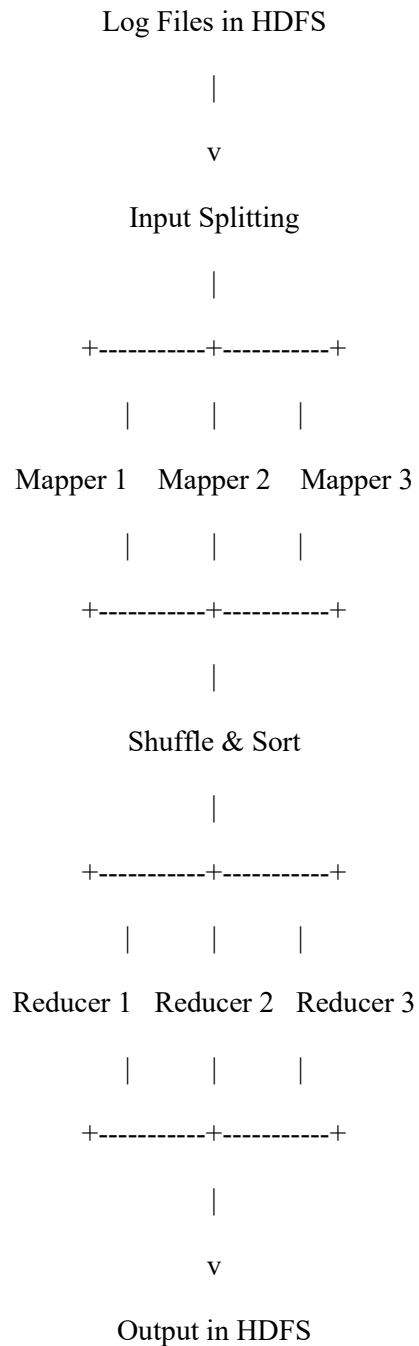
Answer

A large application can generate millions of log records containing timestamps, user IDs, IP addresses, URLs, errors, and response times. MapReduce can process these logs in parallel.

The major constraints are:

- Processing must finish within **20 minutes**
- The number of cluster nodes is limited
- Work must be distributed efficiently
- The system should avoid unnecessary data movement

1. Proposed MapReduce Workflow



2. Input Storage

Log files are first stored in HDFS.

For example:

/logs/day1/

/logs/day2/

/logs/day3/

HDFS divides large files into blocks. Map tasks process these blocks in parallel.

3. Mapper Design

Each Mapper reads individual log records.

For example:

2026-08-24 /login 200

2026-08-24 /home 200

2026-08-24 /payment 500

The Mapper can generate intermediate key-value pairs such as:

/login → 1

/home → 1

/payment → 1

For error analysis:

500 → 1

404 → 1

200 → 1

4. Combiner

A **Combiner** should be used when the operation permits it.

For example, instead of sending:

/login → 1

/login → 1

/login → 1

over the network, the Combiner can produce:

/login → 3

This reduces intermediate data and network traffic.

5. Shuffle and Sort

The MapReduce framework groups intermediate values having the same key.

For example:

Mapper 1: /login → 10

Mapper 2: /login → 15

Mapper 3: /login → 20

They are grouped as:

/login → [10,15,20]

6. Reducer

The Reducer aggregates the values:

/login → 10 + 15 + 20

→ 45

The final results are stored in HDFS.

7. Task Distribution with Limited Nodes

Since the cluster has limited nodes, the number of Map and Reduce tasks should be matched to available CPU and memory resources.

Example:

Node 1 → Mapper 1 + Reducer 1

Node 2 → Mapper 2 + Reducer 2

Node 3 → Mapper 3 + Reducer 3

The ResourceManager/YARN allocates available resources to tasks.

8. Meeting the 20-Minute Constraint

To meet the 20-minute batch window:

1. Store logs in HDFS before processing.
2. Use suitable HDFS block sizes.
3. Maximize parallel Map tasks within available resources.
4. Use combiners to reduce network traffic.
5. Partition reducer workloads evenly.
6. Avoid unnecessary replication or intermediate data movement.
7. Process only the required log fields.
8. Monitor task execution and identify slow tasks.

9. Handling Data Skew

If one key occurs much more frequently than others, a reducer can become overloaded.

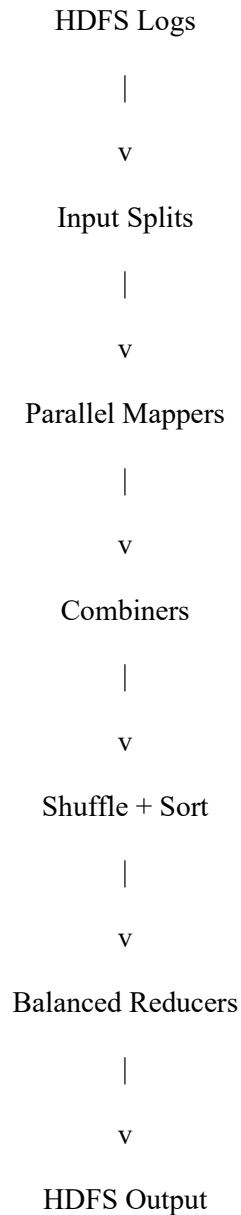
For example:

/login → 90% of records

This creates a bottleneck.

The partitioning strategy should therefore distribute keys as evenly as possible.

10. Final Design



Conclusion

The proposed MapReduce design uses **parallel input processing, combiners, balanced partitioning, and efficient task distribution** to process log data within the 20-minute batch window. Limited nodes are used efficiently by assigning multiple tasks according to available CPU and memory resources.

Q3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

Answer

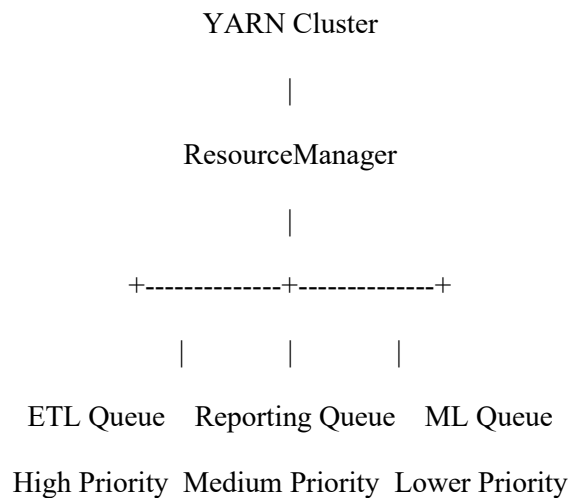
YARN provides resource management for Hadoop applications. In this scenario, three types of workloads must share the cluster:

- ETL jobs
- Reporting jobs
- Machine learning jobs

The scheduler must provide **fairness**, give **higher priority to ETL**, and recover from **node failures**.

1. Proposed Queue Structure

A queue-based policy can be designed as:



2. ETL Queue

ETL jobs should receive higher priority because they prepare data required by downstream analytics.

Example allocation:

ETL Queue → 50%

Reporting Queue → 25%

ML Queue → 25%

These percentages are illustrative and can be adjusted according to workload requirements.

3. Reporting Queue

Reporting workloads receive medium priority.

They should have enough guaranteed resources to prevent starvation.

If ETL activity decreases, unused resources can be temporarily allocated to reporting.

4. Machine Learning Queue

Machine learning jobs can be assigned lower priority because they are often resource-intensive and may run for longer periods.

However, the scheduler should ensure that ML jobs do not permanently starve.

5. Fairness

The scheduler should combine **priority and fair resource sharing**.

For example:

ETL → High priority

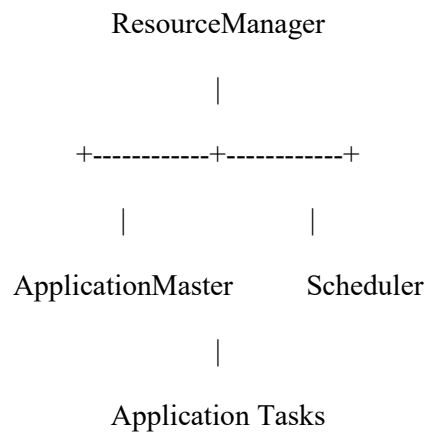
Reporting → Medium priority

ML → Lower priority

Within each queue, multiple users/jobs should receive fair access to resources.

6. ResourceManager

The ResourceManager is responsible for cluster-wide resource allocation.

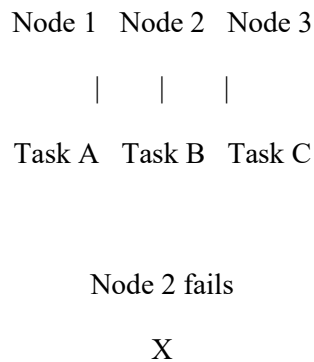


The scheduler decides which applications receive containers.

7. Node Failure Recovery

Suppose Node 2 fails:

Before:



Task B → Reallocated to Node 1/3

The failed task should be detected and rescheduled on a healthy node.

YARN can launch replacement containers for failed tasks.

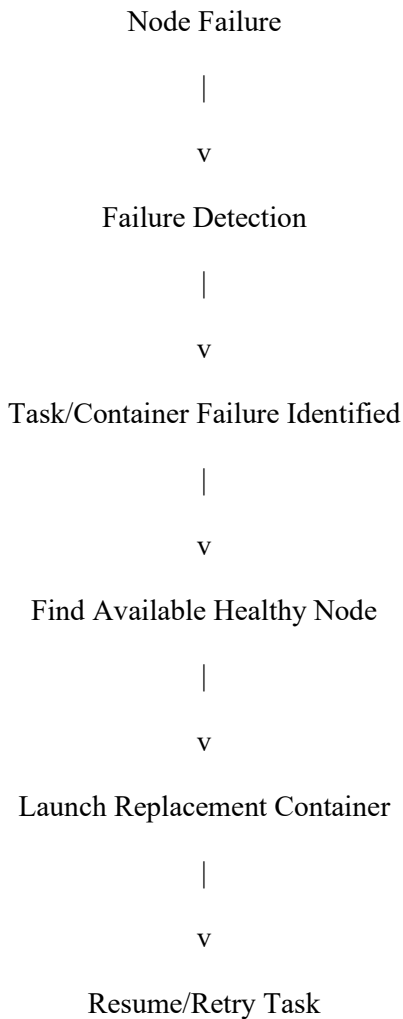
8. Avoiding Starvation

A strict priority-only system could cause ML jobs to wait indefinitely.

Therefore, the policy should include:

- Minimum resource guarantees
- Fair sharing within queues
- Maximum resource limits
- Queue capacity
- Controlled priority
- Preemption only when necessary

9. Recovery Process



10. Proposed Policy

Workload Priority Guaranteed Resources Scheduling

ETL	High	50%	Priority + fair sharing
Reporting	Medium	25%	Fair sharing
ML	Lower	25%	Fair sharing

Conclusion

The proposed YARN policy provides **high priority to ETL while maintaining minimum resources for reporting and machine-learning jobs**. Fair sharing prevents starvation, while automatic task/container rescheduling provides recovery after node failure. This makes the cluster reliable and suitable for mixed workloads.

Q4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

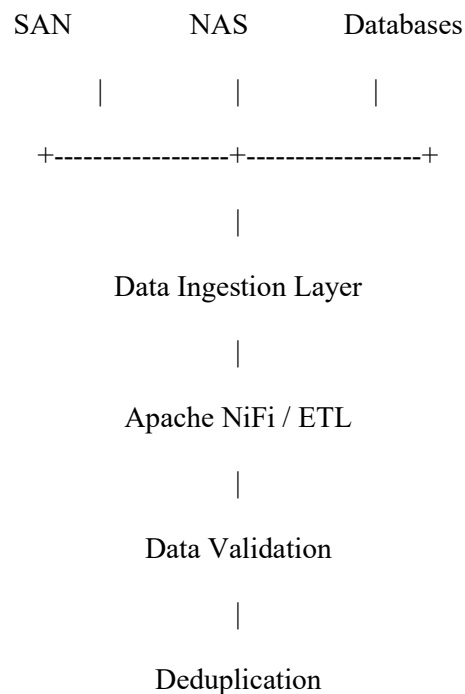
Answer

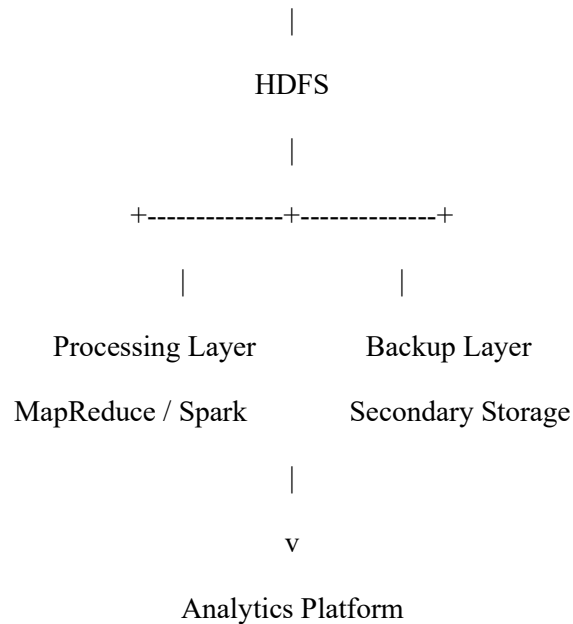
An enterprise may have data distributed across:

- **SAN** – Storage Area Network
- **NAS** – Network Attached Storage
- Operational databases

The goal is to bring this data into a big data platform while maintaining **backup reliability** and minimizing duplicate data.

1. Proposed Architecture





2. SAN Integration

SAN data can be accessed through suitable enterprise connectors or ingestion processes.

The ingestion system should identify:

- File name
- Timestamp
- Source
- File size
- Unique identifier

before storing the data in HDFS.

3. NAS Integration

NAS files can be collected through controlled file-ingestion mechanisms.

Instead of repeatedly copying unchanged files, the system should maintain metadata about previously processed files.

4. Operational Database Integration

Database records can be ingested using:

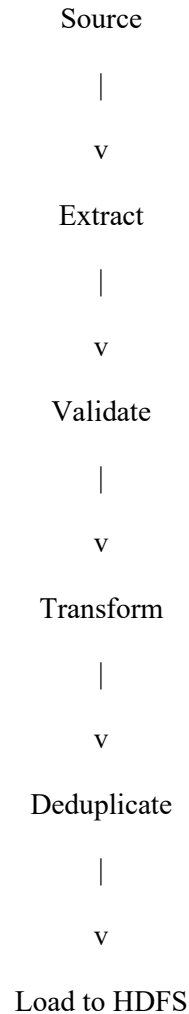
- Incremental extraction
- Change tracking
- Timestamp-based extraction
- ETL/database connectors

Instead of extracting the entire database every time, only new or changed records should be transferred.

5. Apache NiFi / ETL Layer

A centralized ingestion layer controls data movement.

Example:



This provides controlled and traceable data flow.

6. Deduplication

To minimize duplication, each incoming object or record can be associated with a unique identifier.

For example:

Source ID + Record ID + Timestamp

For files, metadata or content hashes can help identify duplicate files.

If a duplicate is detected:

New Data
|

Check Existing Data

|
+--+--+

| |
Yes No

| |
Skip Store

7. HDFS Storage

Validated data is stored in HDFS.

A logical structure could be:

/data/

/san/

/nas/

/database/

This preserves source information and simplifies management.

8. Backup Reliability

Important datasets should have reliable backup and recovery mechanisms.

The architecture should provide:

- HDFS replication
- Backup copies of critical data
- Metadata protection
- Periodic backup validation
- Recovery procedures

9. Data Governance

Metadata should record:

- Data source
- Ingestion time
- Data owner
- Processing status

- Version
- Validation status

This helps prevent repeated ingestion and improves traceability.

10. Constraint-Based Design

The architecture satisfies the constraints as follows:

Constraint	Solution
SAN ingestion	Controlled ingestion pipeline
NAS ingestion	File-based ingestion
Database ingestion	Incremental extraction
Minimal duplication	IDs, metadata and deduplication
Backup reliability	HDFS replication + backup
Centralized management	NiFi/ETL layer
Scalability	HDFS distributed storage

Conclusion

The proposed architecture uses **SAN/NAS/database connectors, Apache NiFi or ETL pipelines, validation, deduplication, HDFS, replication, and backup mechanisms**. Incremental database extraction and duplicate detection reduce unnecessary storage and network usage while maintaining reliable backups.

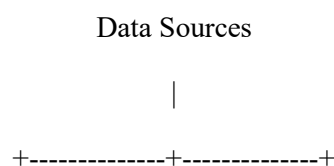
Q5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

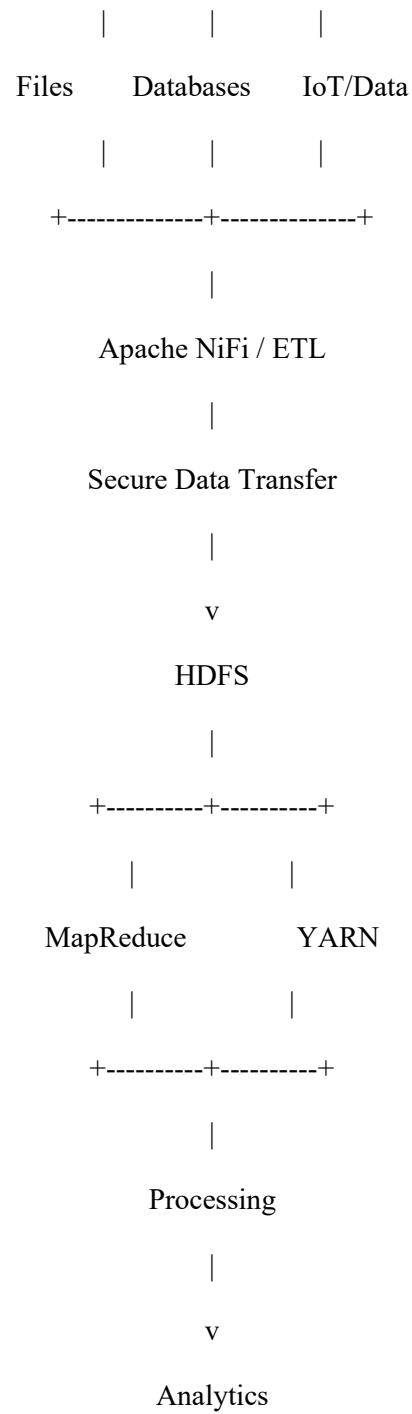
Answer

An end-to-end Hadoop ecosystem should support secure data ingestion, distributed storage, processing, resource management, and analytics. The design must satisfy three main constraints:

1. **Secure data movement**
2. **High availability**
3. **Integration with Apache NiFi or ETL pipelines**

1. Proposed Architecture





2. Data Ingestion

Apache NiFi can act as the ingestion and flow-management layer.

It receives data from different sources and performs:

- Data collection
- Routing
- Filtering
- Transformation

- Validation
- Loading

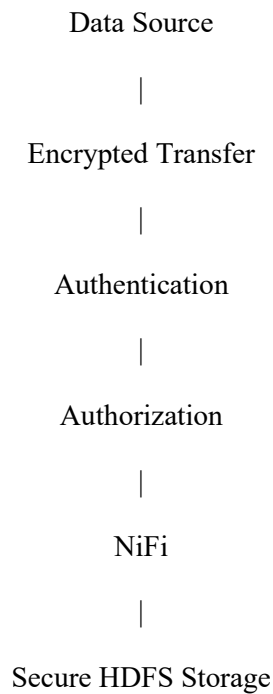
3. Secure Data Movement

Data should be protected while moving between components.

Security mechanisms can include:

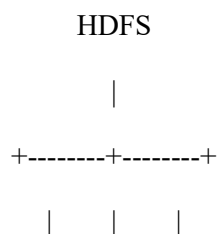
- Encryption during transmission
- Authentication
- Authorization
- Secure protocols
- Access control
- Audit logging

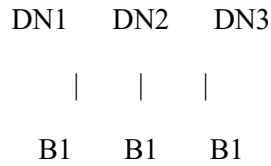
A simplified flow is:



4. HDFS Storage

After ingestion, data is stored in HDFS.



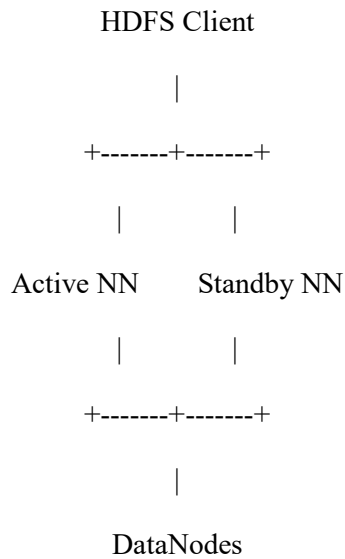


Replication protects against DataNode failures.

5. High Availability

High availability should be provided at critical Hadoop components.

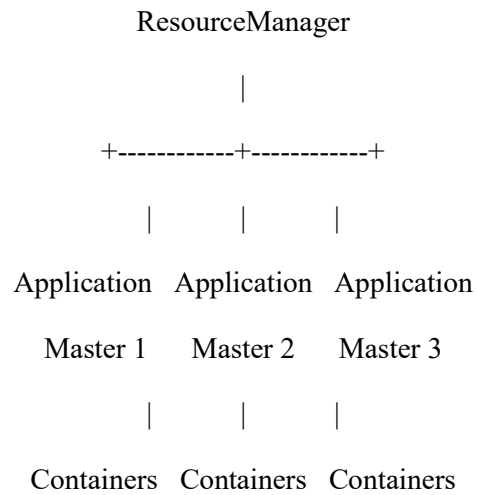
For HDFS:



If the Active NameNode fails, the Standby NameNode can take over.

6. YARN Resource Management

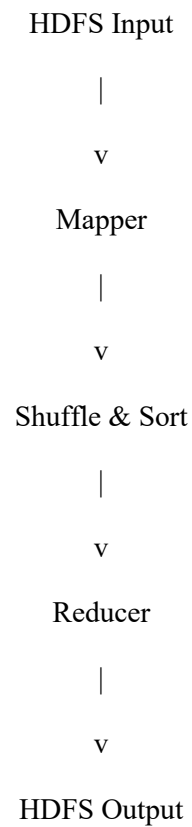
YARN manages processing resources.



This allows multiple processing applications to share the cluster.

7. MapReduce Processing

MapReduce can process large datasets:

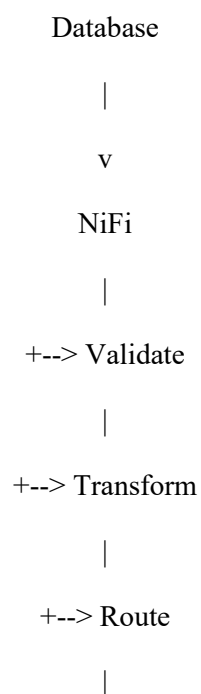


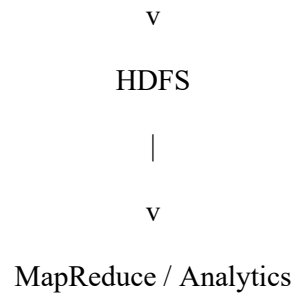
This provides distributed processing across multiple nodes.

8. NiFi Integration

NiFi can continuously ingest data into Hadoop.

Example:



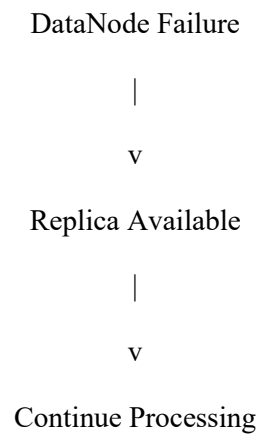


The flow can also include error handling so that invalid data is routed separately instead of contaminating the main dataset.

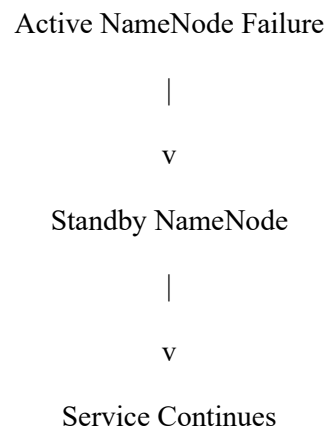
9. Fault Recovery

The architecture should provide recovery at multiple levels:

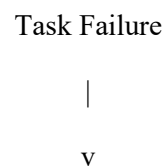
DataNode failure:



NameNode failure:



Processing task failure:



YARN Detects Failure

|

v

Task Rescheduled

10. Constraint Satisfaction

Constraint	Design Solution
Secure data movement	Authentication, authorization and encrypted transfer
High availability	Active/Standby NameNodes and replicated HDFS blocks
Data ingestion	Apache NiFi / ETL
Distributed processing	MapReduce
Resource management	YARN
Fault recovery	Replication and task rescheduling
Scalability	Additional DataNodes

Conclusion

The proposed end-to-end Hadoop ecosystem integrates **Apache NiFi/ETL, HDFS, YARN, and MapReduce** into a unified architecture. Secure data transfer protects information during ingestion, HDFS replication and NameNode high availability improve system reliability, and YARN provides controlled resource management. Thus, the architecture satisfies the required **security, high availability, and data-pipeline integration constraints**.